

Full-Stack E-Commerce Web Application

Technical System Documentation & Architecture Design

Project Type: React & Node.js E-Commerce Platform | **Target Scale:** 1,000,000 Products & 500,000 Users | **Author:** Huzaifa Khan

1. System Executive Overview

This technical documentation outlines the system architecture, frontend structure, database relational model, performance scaling strategies, and security protocols for an enterprise-level E-Commerce web application. The platform is designed with a modern React.js client interface communicating with a scalable Node.js/RDBMS backend built to serve **1,000,000 product items** and **500,000 registered users** with sub-second response times.

2. System Architecture & Tech Stack

Layer	Technology / Framework	Core Responsibility
Frontend Client	React.js, React Router, Context API / Redux	Single-Page Application (SPA), dynamic UI components, stateful cart management, client-side route guards.
Application API	Node.js, Express.js (REST API)	Business logic execution, JWT authentication middleware, input validation, order processing.
Database Layer	PostgreSQL / MySQL (Relational DBMS)	ACID-compliant storage, structured relational schemas, indexing, foreign key constraints.
Security Middleware	Bcrypt, DOMPurify, Helmet, Express-Rate-Limit	Credential hashing, XSS sanitization, rate-limiting, security headers, CORS enforcement.

3. Database Schema & Data Modeling (1M Products Scale)

To support high concurrent transactions without performance bottlenecking, the relational database is organized up to **Third Normal Form (3NF)**.

Entity Table	Field Specifications	Keys & Constraints
users	user_id (BIGINT / UUID) email (VARCHAR 255) password_hash (VARCHAR 255) full_name (VARCHAR 100) role (VARCHAR 20 — 'customer', 'admin') created_at (TIMESTAMP)	PRIMARY KEY user_id UNIQUE INDEX email
categories	category_id (BIGINT) name (VARCHAR 100) slug (VARCHAR 100) parent_category_id (BIGINT NULL)	PRIMARY KEY category_id FOREIGN KEY parent_category_id → categories.category_id
products	product_id (BIGINT) title (VARCHAR 255) slug (VARCHAR 255)	PRIMARY KEY product_id FOREIGN KEY category_id → categories.category_id

Entity Table	Field Specifications	Keys & Constraints
	description (TEXT) price (DECIMAL 10,2) stock_quantity (INT) category_id (BIGINT) created_at (TIMESTAMP)	UNIQUE INDEX slug COMPOSITE INDEX (category_id, price)
orders	order_id (BIGINT) user_id (BIGINT) status (VARCHAR 30) total_amount (DECIMAL 10,2) created_at (TIMESTAMP)	PRIMARY KEY order_id FOREIGN KEY user_id → users.user_id
order_items	order_item_id (BIGINT) order_id (BIGINT) product_id (BIGINT) quantity (INT) unit_price (DECIMAL 10,2)	PRIMARY KEY order_item_id FOREIGN KEY order_id → orders.order_id FOREIGN KEY product_id → products.product_id

4. Performance Scaling Strategies

4.1 Database Indexing (B-Tree Data Structures)

Scanning 1,000,000 records sequentially yields an execution complexity of $O(N)$. Implementing balanced B-Tree indexes reduces lookup time complexity to $O(\log N)$:

- **Foreign Key Indexing:** Speed up multi-table relational JOIN operations between products and categories, as well as orders and users.
- **Composite Query Indexes:** Adding a joint index on (category_id, price) allows fast filtering and sorting of product listings without requiring in-memory database sorting.
- **Full-Text Search Indexing:** Enables direct, fast text match queries across product titles and descriptions for search bar operations.

4.2 Normalization (3NF)

Organizing schema structure up to 3NF eliminates data duplication across large data tables:

- **Storage Savings:** Prevents repeating category names inside 1,000,000 product records, conserving significant disk storage.
- **Write/Update Acceleration:** Updates to category attributes require altering a single row in categories rather than updating hundreds of thousands of individual product rows.

Caching Strategy: Highly requested read queries (such as hot product catalogs or category lists) can be cached in Redis in-memory storage to decrease database load by up to 80%.

5. Security Vulnerabilities & Mitigation Controls

- ### 1. Cross-Site Scripting (XSS) Mitigation

Mechanism: Malicious script injection into input fields rendering in client browsers.

Controls: React automatically escapes variables rendered in JSX. Where rich text rendering is needed, inputs are parsed via DOMPurify. Authentication JWT tokens are saved exclusively in HttpOnly, Secure cookies to prevent client script access.
- ### 2. Cross-Site Request Forgery (CSRF) Mitigation

Mechanism: Unauthorized action execution via cross-site session requests.

Controls: Cookies are set with SameSite=Strict or SameSite=Lax policies. Backend APIs enforce strict CORS origin validation, blocking requests originating from unauthorized domains.
- ### 3. SQL Injection (SQLi) Mitigation

Mechanism: Altering raw SQL query logic through unescaped user inputs.

Controls: All database operations use Parameterized Queries or Object-Relational Mappers (Prisma / Sequelize). Raw string concatenation for SQL statements is strictly prohibited across all endpoints.
- ### 4. Authentication & Rate-Limiting Controls

Mechanism: Automated brute-force guessing attacks on login routes.

Controls: User passwords are encrypted using Bcrypt with a salt factor of 10+. Authentication endpoints (/api/login) feature rate-limiting middleware restricting requests per IP address.

6. Core REST API Routes

Method	Endpoint Route	Description / Function
POST	/api/auth/register	Registers a new user account with hashed password storage.
POST	/api/auth/login	Authenticates credentials and returns an HttpOnly JWT cookie.
GET	/api/products	Fetches paginated, indexed products list with category filtering.
GET	/api/products/:slug	Fetches detailed product item metadata via indexed slug lookup.
POST	/api/orders	Creates a new customer order and associated normalized order items.